

Practical - 9

1. File Copy using Low-Level I/O

```
harshika@siri:~$ gedit file_copy.c
** (gedit:546): WARNING **: 09:01:32.696: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:546): WARNING **: 09:01:32.696: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~$ gedit source.txt
** (gedit:762): WARNING **: 09:02:25.121: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:762): WARNING **: 09:02:25.121: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~$ gcc -Wall -g file_copy.c -o file_copy
harshika@siri:~$ ./file_copy
=====
PRACTICAL 9 - FILE COPY
=====
Enter source file name: source.txt
-----
1. LOW-LEVEL SYSTEM CALL COPY
-----
Copy completed successfully.
Destination: low_level_copy.txt
Time taken: 0.000311 seconds
-----
2. STANDARD LIBRARY COPY
-----
Copy completed successfully.
Destination: standard_copy.txt
Time taken: 0.000533 seconds
-----
PERFORMANCE COMPARISON
=====
Low-level I/O time      : 0.000311 seconds
Standard library time   : 0.000533 seconds
=====
```

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <fcntl.h>
4#include <unistd.h>
5#include <sys/stat.h>
6#include <time.h>
7#include <errno.h>
8#include <string.h>
9
10#define BUFFER_SIZE 4096
11
12/* =====
13   LOW-LEVEL FILE COPY
14   Uses:
15   open()
16   read()
17   write()
18   lseek()
19   close()
20   ===== */
21int low_level_copy(const char *source, const char *destination)
22{
23    int source_fd;
24    int destination_fd;
25    char buffer[BUFFER_SIZE];
26    ssize_t bytes_read;
27    ssize_t bytes_written;
28
29    /* Open source file for reading */
30    source_fd = open(source, O_RDONLY);
31    if (source_fd == -1)
32    {
33        perror("Error opening source file");
34        return -1;
35    }
36
37    /* Create/open destination file */
38    destination_fd = open(
39        destination,
40        O_WRONLY | O_CREAT | O_TRUNC,
41        0644
42    );
43    if (destination_fd == -1)
44    {
45        perror("Error opening destination file");
46        close(source_fd);
47        return -1;
48    }
49
50    while ((bytes_read = read(source_fd, buffer, BUFFER_SIZE)) > 0)
51    {
52        bytes_written = write(destination_fd, buffer, bytes_read);
53        if (bytes_written < bytes_read)
54        {
55            perror("Error writing to destination file");
56            close(source_fd);
57            close(destination_fd);
58            return -1;
59        }
60    }
61    close(source_fd);
62    close(destination_fd);
63    return 0;
64}
```

2. I/O Redirection

```
harshika@siri:~$ gedit io_redirect.c
** (gedit:1436): WARNING **: 09:11:41.009: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:1436): WARNING **: 09:11:41.010: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~$ gcc -Wall -g io_redirect.c -o io_redirect
harshika@siri:~$ ./io_redirect
=====
I/O REDIRECTION USING dup2()
=====
[PARENT PROCESS]
Parent PID: 1747
Child PID: 1748
Waiting for child...

[CHILD PROCESS]
Child PID: 1748
Child completed.
Child exit status: 0

Output has been redirected to:
output.txt
harshika@siri:~$ gedit io_redirect.c
** (gedit:1758): WARNING **: 09:13:29.804: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:1758): WARNING **: 09:13:29.805: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
```

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <fcntl.h>
4#include <unistd.h>
5#include <sys/stat.h>
6#include <string.h>
7
8int main()
9{
10    int input_fd;
11    int output_fd;
12
13    pid_t pid;
14
15    printf("===== \n");
16    printf("I/O REDIRECTION USING dup2() \n");
17    printf("===== \n");
18
19    /* CREATE INPUT FILE
20       ===== */
21    input_fd = open(
22        "input.txt",
23        O_WRONLY | O_CREAT | O_TRUNC,
24        0644
25    );
26    if (input_fd == -1)
27    {
28        perror("input.txt");
29        return 1;
30    }
31
32    const char *text =
33        "Hello from Practical 9!\n"
34        "This data is coming from input.txt.\n"
35        "Standard input will be redirected using dup2()!\n";
36
37    write(input_fd, text, strlen(text));
38    close(input_fd);
39
40    /* OPEN INPUT FILE FOR READING
41       ===== */
42    input_fd = open(
43        "input.txt",
44        O_RDONLY
45    );
46    if (input_fd == -1)
47    {
48        perror("input.txt");
49        return 1;
50    }
51
52    pid = fork();
53    if (pid < 0)
54    {
55        perror("fork");
56        return 1;
57    }
58    if (pid == 0)
59    {
60        /* CHILD PROCESS */
61        dup2(input_fd, 0);
62        printf("Child process started. Reading from input.txt.\n");
63        while ((c = getc(stdin)) != EOF)
64        {
65            putchar(c);
66        }
67        printf("Child process completed.\n");
68        exit(0);
69    }
70    else
71    {
72        /* PARENT PROCESS */
73        printf("Parent process completed.\n");
74        exit(0);
75    }
76}
```